

Phishio: Detecção Colaborativa de Phishing em Tempo Real via Abordagem Híbrida de Recuperação de Informação e *Crowdsourcing**

Phishio: Real-Time Collaborative Phishing Detection through a Hybrid Information Retrieval and Crowdsourcing Approach

Pedro Henrique Gomes Lückeroth¹
Lucas dos Santos Lima²
Sinaide Nunes Bezerra³

Resumo

O *phishing* persiste como uma das ameaças cibernéticas mais críticas ao explorar a confiança do usuário para o roubo de credenciais. As defesas tradicionais, baseadas em listas negras estáticas, são ineficazes contra a rápida rotatividade de URLs maliciosas em ataques *zero-hour*. É proposto neste trabalho o desenvolvimento de um sistema de detecção em tempo real, composto por uma extensão de navegador e uma API de análise em nuvem. A metodologia adotada combina de forma híbrida técnicas de Recuperação de Informação em especial o Modelo Vetorial com ponderação TF-IDF e Similaridade do Cosseno aplicada a uma base de dados proprietária, integrada a um módulo de validação colaborativa (*crowdsourcing*). A arquitetura utiliza o Google Compute Engine para o processamento vetorial e o *Google Firestore* para o gerenciamento da reputação de URLs baseada no consenso da comunidade. A solução proposta visa reduzir a latência na identificação de novas ameaças e minimizar a taxa de falsos positivos através do *feedback* contínuo dos usuários.

Palavras-chave: Phishing. Recuperação de Informação. Crowdsourcing. Segurança Web. Detecção em Tempo Real.

*Trabalho de conclusão de curso, Sistemas de Informação, Unidade São Gabriel

¹Sistemas de Informação, Brasil– 1213227@sga.pucminas.br

²Sistemas de Informação, Brasil– lucas.lima.1129309@sga.pucminas.br

³Instituto de Ciências Exatas e de Informática da PUC Minas, Brasil– sinaide@pucminas.br

Abstract

Phishing remains one of the most critical cyber threats, exploiting user trust to steal credentials. Traditional defenses based on static blacklists are ineffective against the rapid turnover of malicious URLs in zero-hour attacks. This work proposes the development of a real-time detection system composed of a browser extension and a cloud-based analysis API. The methodology adopts a hybrid approach combining Information Retrieval techniques, specifically the Vector Space Model with TF-IDF weighting and Cosine Similarity, applied to a proprietary database, integrated with a collaborative validation module (crowdsourcing). The architecture utilizes Google Compute Engine for vector processing and *Google Firestore* for URL reputation management based on community consensus. The proposed solution aims to reduce latency in identifying new threats and minimize false positive rates through continuous user feedback.

Keywords: Phishing. Information Retrieval. Crowdsourcing. Web Security. Real-Time Detection.

1 INTRODUÇÃO

A relevância deste estudo é justificada pelo impacto financeiro e operacional dos ataques de engenharia social. Segundo o relatório *Cost of a Data Breach* da IBM (2025), o *phishing* figura de maneira consistente como um dos vetores de acesso inicial mais comuns e custosos para violações de dados corporativos. Corroborando esse cenário, o Federal Bureau of Investigation (2024) aponta o *phishing* como o crime cibernético líder em número de vítimas, gerando perdas bilionárias anualmente.

Apesar do contínuo aprimoramento dos sistemas de detecção, a eficácia dos ataques reside na velocidade. O relatório da Verizon (2024) revela que o tempo mediano para que um usuário seja comprometido após abrir um e-mail de *phishing* é inferior a 60 segundos. Essa janela crítica expõe a fragilidade das listas de bloqueio (*blacklists*) estáticas, que não conseguem acompanhar a volatilidade das ameaças.

Adicionalmente, os atacantes têm migrado suas operações para infraestruturas legítimas para evadir detecções baseadas em reputação de rede. Dados compilados por Martínez (2025), baseados em inteligência da Netcraft, indicam que cerca de 77% dos sites de *phishing* utilizam serviços de nuvem pública para se hospedar. Essa tática, combinada com o ciclo de vida extremamente curto das páginas fraudulentas, muitas vezes ativas por poucas horas, conforme a análise realizada por Kolesnikov e Cuzzo (2021), exige a implementação de mecanismos de detecção em tempo real baseados em análise de conteúdo.

Uma vez que a interação do usuário com páginas fraudulentas ocorre muitas vezes através do navegador web, a integração de ferramentas de segurança diretamente nesse ambiente é estratégica. A consolidação do padrão *Manifest V3* para extensões do Google Chrome corrobora essa premissa ao exigir arquiteturas mais seguras, com foco na privacidade do usuário e no baixo consumo de recursos computacionais.

Diante desse contexto, o objetivo principal deste trabalho é desenvolver uma solução híbrida para a detecção de *phishing* em tempo real. Para alcançar esse propósito, são definidos como objetivos específicos: construir a interface da solução em uma extensão de navegador; implementar uma API em nuvem capaz de realizar o processamento vetorial de textos; e integrar um mecanismo de validação colaborativa (*crowdsourcing*), capaz de unir a capacidade analítica do algoritmo com a inteligência coletiva dos usuários.

A arquitetura proposta assume o caráter de uma solução híbrida devido à convergência de duas frentes distintas de análise. A primeira vertente utiliza o rigor matemático da Recuperação de Informação, por meio do Modelo Vetorial desenvolvido por Baeza-Yates *et al.* (1999), para o processamento técnico do conteúdo das páginas. A segunda vertente estabelece uma camada social baseada no *crowdsourcing*, na qual o consenso e o *feedback* dos usuários refinam a classificação final. Essa integração busca mitigar as limitações de sistemas puramente algorítmicos e reduzir a dependência de listas de bloqueio estáticas.

Este artigo está estruturado da seguinte forma. A Seção 2 apresenta o referencial teórico, no qual se inclui a fundamentação teórica sobre *phishing*, recuperação de informação e *crowd-*

sourcing. A Seção 3 descreve a metodologia utilizada no desenvolvimento do sistema proposto, detalhando a arquitetura e os procedimentos de validação experimental. A Seção 4 aborda a implementação técnica e a infraestrutura de rede em nuvem. A Seção 5 foca no desenvolvimento do módulo cliente, enquanto a Seção 6 detalha o processamento do *back-end* e a validação do índice. A Seção 7 apresenta os resultados obtidos e a discussão sobre o desempenho do classificador. Por fim, a Seção 8 expõe o cronograma de atividades do projeto.

2 REFERENCIAL TEÓRICO

A fundamentação deste trabalho articula conceitos de segurança da informação, recuperação de informação e usabilidade, estabelecendo a base para uma solução de detecção híbrida. Para a devida compreensão desses pilares. Inicialmente, a exposição do panorama das ameaças de *phishing*, que evidência a evolução técnica dos ataques e as limitações das defesas convencionais. Em seguida, a descrição dos preceitos da Recuperação de Informação e do Modelo Vetorial, a base matemática adotada para a identificação de similaridades textuais. Por fim, a discussão sobre a aplicação do *crowdsourcing* atrelado a princípios de usabilidade justifica a viabilidade de inserir o usuário como agente ativo na validação de segurança.

2.1 O Cenário de Ameaças de Phishing

Segundo a APWG (2025), "o *phishing* é um crime que emprega tanto engenharia social quanto subterfúgios técnicos para roubar dados de identidade pessoal e credenciais de contas financeiras dos consumidores". A evolução dessa ameaça tem sido marcada pela velocidade e pela evasão de defesas convencionais.

Relatórios recentes da indústria indicam que a "janela de oportunidade" para a detecção é crítica. Em Verizon (2024) é apontado que o tempo mediano para o comprometimento de um usuário é inferior a 60 segundos. Esse cenário é agravado pela volatilidade das URLs: Kolesnikov e Cuzzo (2021) demonstra que grande parte das páginas de *phishing* são descartáveis e permanecem ativas apenas pelo tempo necessário para capturar um número específico de vítimas, o que torna ineficazes as abordagens baseadas em listas negras (*blacklists*) que possuem latência de atualização. Além disso, a infraestrutura dos ataques mudou. Dados compilados por Martínez (2025) revelam que atacantes utilizam massivamente serviços de nuvem legítimos para hospedar páginas fraudulentas, o que dificulta bloqueios baseados na reputação de IP ou domínio.

Fette, Sadeh e Tomasic (2007) estabeleceram, em estudo seminal, as características estruturais (*features*) que permitem a identificação automatizada de *phishing*, como a idade do domínio, a quantidade de pontos na URL e a presença de formulários inseguros. Neste estudo são apresentadas heurísticas vitais para a modelagem de classificadores modernos.

2.2 Recuperação de Informação e Modelo Vetorial

Para analisar o conteúdo das páginas em tempo real, este trabalho utiliza conceitos de Recuperação de Informação (RI). Segundo Baeza-Yates *et al.* (1999), o Modelo Vetorial permite representar documentos e consultas como vetores em um espaço multidimensional.

Neste modelo, a importância de um termo é calculada através do esquema de ponderação TF-IDF (*Term Frequency-Inverse Document Frequency*), que penaliza termos muito comuns (como artigos e preposições) e valoriza termos raros e discriminantes. A similaridade entre uma página suspeita e um documento conhecido (assinatura de phishing) é calculada pelo cosseno do ângulo entre seus vetores:

$$\text{sim}(d_j, q) = \frac{\vec{d}_j \cdot \vec{q}}{|\vec{d}_j| \times |\vec{q}|} \quad (1)$$

Fonte: Baeza-Yates *et al.* (1999, p. 27).

Zhang, Hong e Cranor (2007) validaram a aplicação dessa teoria na segurança com o sistema CANTINA, ao demonstrar que o conteúdo textual de uma página, processado via TF-IDF, é um forte indicador de sua legitimidade ou intenção maliciosa.

2.3 Crowdsourcing e Usabilidade na Segurança

A abordagem colaborativa, ou *crowdsourcing*, permite utilizar a inteligência coletiva para refinar sistemas de detecção. Em segurança, isso cria um mecanismo de "humano no ciclo" (*human-in-the-loop*), no qual usuários experientes podem sinalizar falsos positivos que algoritmos puramente matemáticos não conseguem discernir.

Para que essa colaboração seja efetiva, a interface deve seguir princípios de usabilidade. Nielsen (1994) preconiza que o sistema deve manter o usuário informado sobre seu estado e permitir a recuperação de erros. No contexto de uma extensão de segurança, isso significa que alertas devem ser claros e o mecanismo de denúncia deve exigir esforço cognitivo mínimo, de forma a evitar a interrupção desnecessária da navegação.

3 METODOLOGIA

A presente pesquisa é de natureza aplicada, uma vez que visa resolver um problema prático de segurança da informação por meio do desenvolvimento de um artefato de software (extensão de navegador e API). Quanto à abordagem do problema, o estudo é quantitativo, visto que utiliza métricas numéricas para avaliar o desempenho do sistema, tais como a latência de processamento e a taxa de acurácia na classificação de URLs (mensurada via matriz de confusão). Do ponto de vista dos procedimentos técnicos, o trabalho consiste em uma pesquisa

experimental, visto que o artefato está submetido a testes em um ambiente controlado, que utiliza uma base mista de URLs legítimas e maliciosas para validar as hipóteses de detecção em tempo real. A metodologia de desenvolvimento segue uma abordagem iterativa, dividida nas etapas de construção do *dataset*, implementação da arquitetura em nuvem e desenvolvimento do módulo cliente.

Esta seção aborda, inicialmente, a arquitetura do sistema proposto, que estabelece os três módulos centrais da solução. Na sequência, o fluxo de validação híbrida é detalhado para ilustrar a interação e o comportamento dinâmico entre esses componentes. Posteriormente, os mecanismos de instrumentação e coleta de dados são descritos; esses mecanismos servem para a mensuração das métricas de desempenho e acurácia. Por fim, os procedimentos metodológicos são listados para organizar o desenvolvimento e a validação experimental dos artefatos desenvolvidos.

3.1 Arquitetura do Sistema Proposto

A infraestrutura tecnológica do projeto detalha o modelo de comunicação e a distribuição das responsabilidades computacionais. A proposta mostra como a arquitetura integra a interface do usuário, o processamento analítico em nuvem e o banco de dados colaborativo, o que possibilita a detecção em tempo real.

A solução mostrada na Figura 1 é estruturada em uma arquitetura Cliente-Servidor distribuída, composta por três módulos principais:

1. **Módulo Cliente (Extensão de Navegador):** Atua como a interface direta com o usuário e o interceptador de eventos de navegação. Este módulo foi otimizado para um fluxo *URL-First*: inicialmente, a extensão envia apenas a URL para uma verificação rápida no servidor. O conteúdo denso (DOM) só é extraído e enviado de maneira assíncrona se a API retornar um *status* "Desconhecido" (*Cache MISS*), minimizando o consumo de banda e a latência de rede em páginas já catalogadas. Além disso, implementa um ciclo de vida resiliente, limpando estados de memória local ao fechar abas para evitar vazamentos de recursos no navegador.
2. **Módulo de Processamento (Back-end API):** Estruturado como uma API *RESTful*⁴ hospedada em ambiente de nuvem, atua como o núcleo orquestrador do fluxo de validação híbrida utilizando FastAPI de Tiangolo (2026). A arquitetura garante o desacoplamento *client-server* e a comunicação estritamente *stateless* com a extensão de navegador, trafegando pacotes de dados via JSON. Ao receber uma requisição, o módulo prioriza a consulta ao *cache* de reputação colaborativa. Apenas em cenários de URLs desconhecidas (potenciais ataques *zero-hour*) ou em URLs que se encontram em estado neutro

⁴O termo REST (*Representational State Transfer*) refere-se ao estilo arquitetural e ao conjunto de restrições teóricas (como comunicação *stateless* e interface uniforme) para sistemas distribuídos criadas por Roy Fielding. O termo *RESTful* é o adjetivo utilizado para qualificar um sistema ou serviço web que implementa e obedece a essas restrições na prática.

(sem o consenso de segurança consolidado pela comunidade), a API aciona o motor de Recuperação de Informação, gerando a representação vetorial do texto e processando a Similaridade do Cosseno para decidir se é uma página segura ou suspeita.

3. **Módulo de Persistência e Colaboração:** Baseado em um banco de dados orientado a documentos (*NoSQL*) em nuvem, gerencia o estado dinâmico das URLs e consolida a inteligência coletiva. O módulo converte as URLs analisadas em identificadores únicos através de *hash* SHA-256, o que garante estabilidade nas operações de leitura e previne falhas com caracteres especiais na estruturação do banco.

Para quantificar a inteligência coletiva, foi implementada uma métrica denominada *Score de Consenso* (S_c), fundamentada no conceito da sabedoria das multidões proposto por Surowiecki (2023) e em sistemas de reputação. Cada interação do usuário na extensão atua como um voto (v), assumindo valores binários opostos: +1 para denúncias de *phishing* e -1 para marcações de site seguro. O *score* de uma determinada URL é definido pela soma algébrica de todos os votos recebidos, conforme a Equação 2:

$$S_c = \sum_{i=1}^n v_i \quad \text{onde} \quad v_i \in \{1, -1\} \quad (2)$$

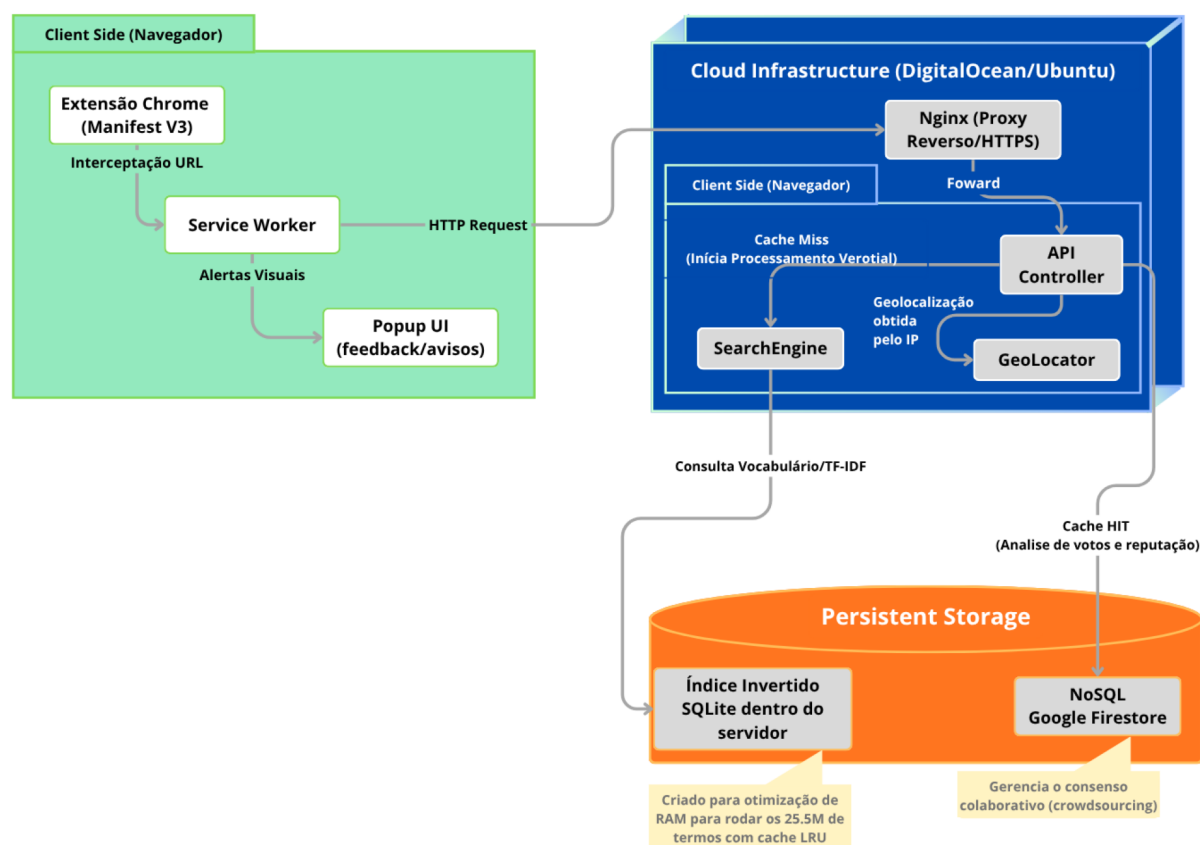
Modelo matemático conhecido como Caminhada Aleatória Simples Unidimensional (*random walk ID*)

Com base no valor atual de S_c , o módulo de persistência gerencia a transição de estados da URL obedecendo a limiares assimétricos de decisão:

- **Estado Confiável** ($S_c < 0$): Se a URL obtém mais votos de segurança do que denúncias, ela é classificada como segura (*safe*) e o veredito é servido diretamente via *cache* global nas próximas requisições.
- **Estado Neutro** ($0 \leq S_c < 15$): A URL possui denúncias, mas o volume não é estatisticamente suficiente para confirmar a ameaça. Neste cenário, o sistema não confia apenas na colaboração, e a API orquestradora delega a classificação da ameaça ao Motor de Recuperação de Informação, que utiliza TF-IDF e Similaridade do Cosseno.
- **Estado Malicioso** ($S_c \geq 15$): Ao atingir um saldo positivo de 15 denúncias, o sistema consolida o consenso da comunidade. A URL assume o *status* de *phishing* e passa a alertar usuários sobre o perigo, dispensando o processamento de conteúdo no *back-end*.

A escolha por limiares assimétricos atua como um mecanismo de defesa contra manipulações da rede (Ataques *Sybil*) e reduz falsos positivos. Para assegurar a integridade dos dados durante esse fluxo contínuo de *crowdsourcing*, a persistência de cada incremento na variável S_c é tratada estritamente por meio de transações atômicas no banco de dados, prevenindo falhas de concorrência (*race conditions*).

Figura 1 – Visão arquitetural do módulo cliente, processamento vetorial e base colaborativa



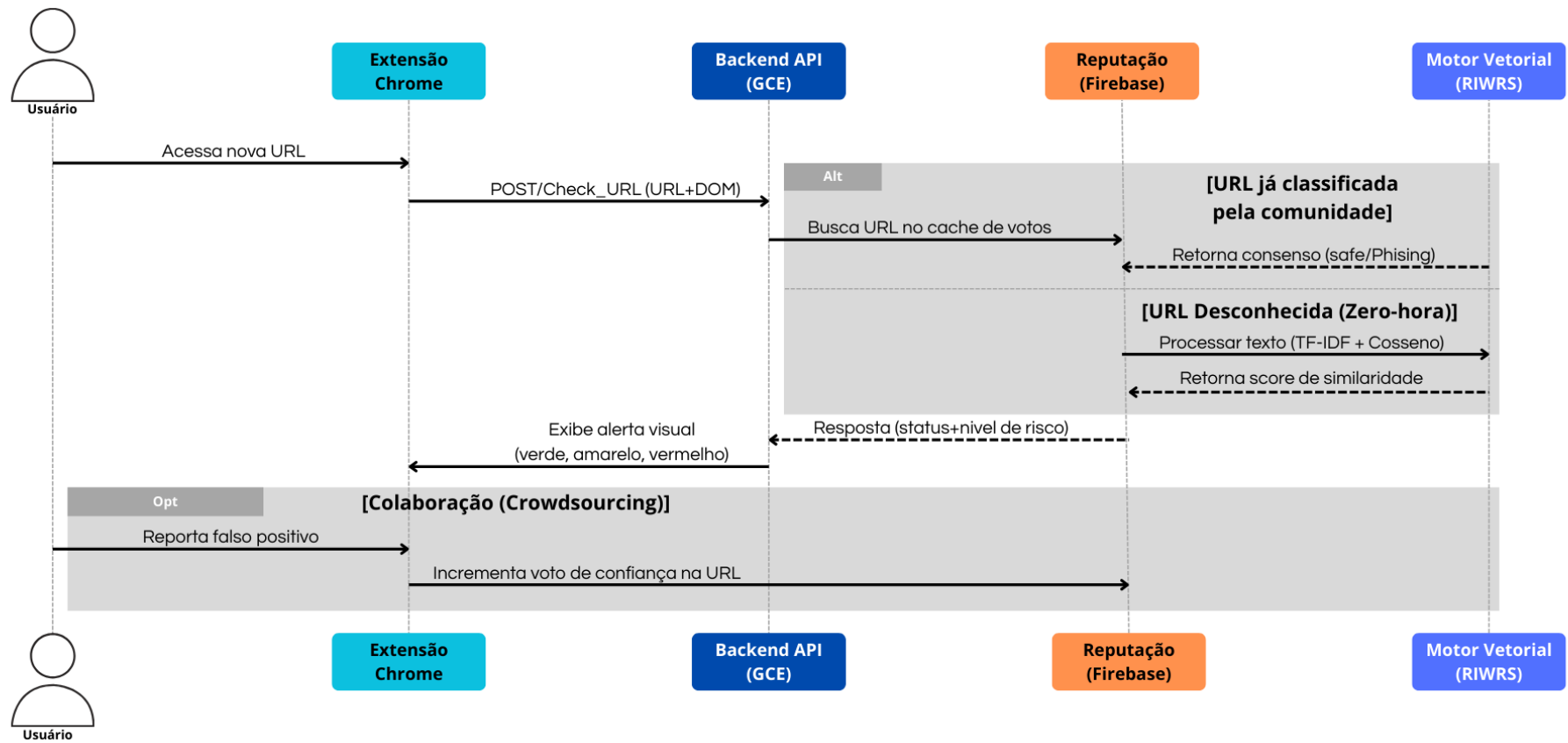
Fonte: Dados da Pesquisa (2026).

3.2 Fluxo de Validação Híbrida

Para compreender a interação dinâmica entre os módulos descritos na arquitetura, a Figura 2 apresenta o diagrama de sequência do sistema. O fluxo foi projetado para otimizar o tempo de resposta e poupar recursos computacionais, fato que fundamenta a natureza híbrida da solução.

Quando o usuário acessa uma nova URL, a extensão intercepta a requisição e envia os dados para a API. Antes de acionar o processamento vetorial (que possui maior custo computacional), o sistema realiza uma verificação no cache de votos colaborativos (Firestore). Se a URL já possuir um consenso formado pela comunidade, o veredito é retornado imediatamente. Apenas em casos de URLs desconhecidas (potenciais ataques *zero-hour*) o motor de Recuperação de Informação é instanciado para calcular a Similaridade do Cosseno com base no texto. Por fim, o ciclo se encerra com a possibilidade de o usuário atuar ativamente ao reportar possíveis falsos positivos para atualizar novamente a base de reputação.

Figura 2 – Diagrama de sequência do fluxo híbrido de validação



Fonte: Dados da Pesquisa (2026).

3.3 Instrumentação e Coleta de Dados

Os mecanismos de monitoramento e armazenamento de dados integrados à solução visam explicitar como o retorno dos usuários e a precisão do algoritmo serão capturados para embasar a avaliação do experimento. O sistema é instrumentado em três níveis de coleta de modo a garantir a observabilidade da eficácia técnica e do impacto social:

- **Métricas de Eficácia e Colaboração (Back-end):** O sistema utiliza o *Google Firestore* para o registo da percentagem de ligações identificadas pela comunidade e a relação entre reportes confirmados e falsos positivos.
- **Engajamento e Escala:** O monitoramento da volumetria de usuários ativos é correlacionado ao crescimento da base de dados. Adicionalmente, a localização relativa via IP permite a análise da concentração geográfica das ameaças identificadas
- **Acurácia e Performance Indireta:** O sistema mantém um registro anônimo das URLs classificadas pelo motor vetorial para o cálculo da matriz de confusão. A performance é verificada via ferramentas de inspeção do navegador (*DevTools*), com foco no tempo de resposta das chamadas à API

3.4 Procedimentos Metodológicos

Neste tópico, o roteiro de execução da pesquisa empírica é delineado. O objetivo é fornecer uma visão cronológica de como a construção do artefato e a validação do sistema ocorrerão, desde a preparação da base de dados inicial até a apuração dos resultados. Diante disso, o desenvolvimento segue as seguintes etapas:

- **Etapa 1 - Preparação do Corpus (Cold Start):** Para mitigar o problema de *Cold Start* e fornecer uma base de conhecimento inicial ao motor vetorial, foi utilizado um *dataset* proprietário composto por aproximadamente 100.000 documentos HTML de páginas de *phishing*. Esse *corpus* foi consolidado a partir de uma coleta realizada anteriormente na disciplina de Recuperação de Informação na Web e Redes Sociais. A coleta original foi baseada em um universo de 457.265 URLs pré-definidas, das quais 105.548 páginas ativas foram capturadas integralmente via *crawler* vertical desenvolvido em *Python*, utilizando *FastAPI*. O processo utilizou uma arquitetura multithreading com 30 *workers* simultâneos para lidar com a alta volatilidade e latência da infraestrutura de *phishing*. Os documentos foram submetidos a um *pipeline* de análise léxica (limpeza de *tags*, remoção de *stopwords* e *stemming*) e indexados através da técnica *SPIMI* (*Single-pass in-memory indexing*), resultando em um índice invertido que serve como referência para os vetores de similaridade do projeto Phishio que também utiliza *FastAPI* ((TIANGOLO, 2026)) e *SQLite Development Team* (2026).

- **Etapa 2 - Desenvolvimento da API:** Migração do código de busca local para um serviço web escalável, otimizando o carregamento do índice, superior a 4,5 GB, na memória RAM da instância em nuvem.
- **Etapa 3 - Integração Híbrida:** Implementação da lógica que consulta primeiro a base colaborativa (Firestore) e, em caso de ausência de registro, aciona o motor vetorial (Compute Engine) para possibilitar performance.
- **Etapa 4 - Validação Experimental:** Realização de testes de acurácia utilizando uma base de controle mista (URLs legítimas e maliciosas) e medição da latência média de resposta da API para garantir que não haja degradação perceptível na navegação do usuário (meta < 500ms).

4 IMPLEMENTAÇÃO TÉCNICA E INFRAESTRUTURA

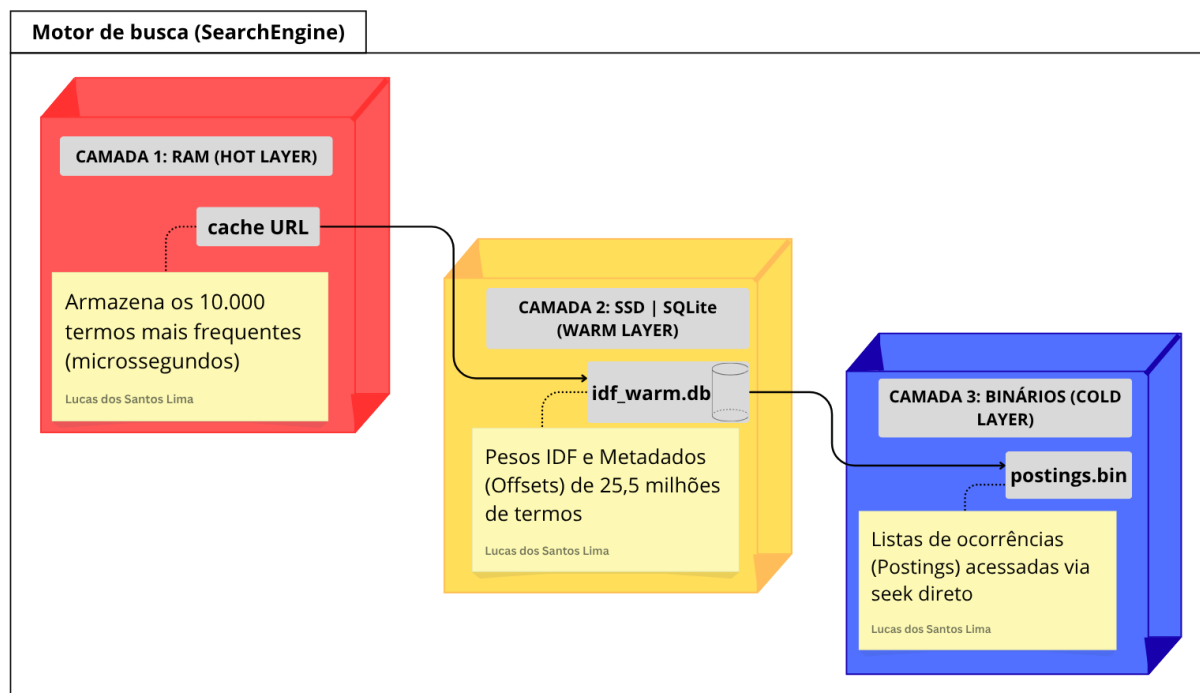
Nesta seção, são descritas as evoluções na arquitetura do sistema Phishio, com foco na otimização de recursos computacionais e no endurecimento (*hardening*) da segurança na camada de comunicação entre a extensão e o motor de análise. A transição de um protótipo local para uma solução operante em ambiente real exigiu a superação de desafios críticos de infraestrutura.

4.1 Otimização de Memória via Armazenamento Híbrido

A escala do índice invertido, que compreende aproximadamente 25,5 milhões de termos e metadados estatísticos, tornou inviável a carga integral desses artefatos em memória RAM no ambiente de nuvem que contava com apenas 4 GB de memória RAM disponível. Para mitigar o encerramento de processos pelo sistema operacional (*Out of Memory Killer*), os dados foram migrados de arquivos *JSON* para o banco de dados relacional *JSON*.

Essa estratégia de armazenamento em camadas (*Tiered Storage*) consolidou o vocabulário e os pesos IDF em uma única tabela indexada. O motor vetorial realiza consultas pontuais ao SSD via *JSON* e utiliza uma camada de *cache* LRU (*Least Recently Used*) para os 10.000 termos mais frequentes. Com essa implementação, a ocupação de RAM foi reduzida de 4 GB para menos de 500 MB.

Figura 3 – Diagrama de Arquitetura de Dados (Tiered Storage)



Fonte: Dados da Pesquisa (2026).

4.2 Infraestrutura de Rede e Segurança

A infraestrutura utiliza o servidor Nginx como um *Proxy Reverso* à frente da aplicação FastAPI ((TIANGOLO, 2026)). Essa camada gerencia as conexões externas e encaminha as requisições internamente para a porta local 8000, o que garante o isolamento do *back-end*. A comunicação entre a extensão e a API ocorre estritamente sobre o protocolo HTTPS, com certificados emitidos pela autoridade *Let's Encrypt*.

Para garantir a integridade dos votos no mecanismo de *crowdsourcing*, a API implementa uma camada de defesa contra *IP Spoofing*. O sistema valida se as requisições provêm de *proxies* reversos confiáveis (*TRUSTED_PROXIES*) antes de processar os cabeçalhos de encaminhamento, registrando tentativas de fraude em logs de auditoria.

Complementarmente, a geolocalização dos acessos foi migrada de uma API externa para um motor local baseado no banco de dados *MaxMind GeoLite2*. Esta transição eliminou a latência de chamadas de rede externas e reforçou a privacidade dos usuários, uma vez que os endereços IP não são mais compartilhados com serviços de terceiros durante a análise.

4.3 Desafios de Infraestrutura e Falsos Positivos

O desenvolvimento do Phishio enfrentou o desafio de processar ameaças em tempo real com recursos restritos. A volatilidade das URLs de *phishing*, que muitas vezes permanecem

ativas por poucas horas, exige que a detecção seja proativa e resiliente.

Um desafio central reside na ocorrência de falsos positivos, nos quais algoritmos puramente matemáticos podem falhar ao discernir páginas legítimas de fraudulentas. Essa limitação reforça a justificativa para a criação da base colaborativa (*crowdsourcing*). Na arquitetura proposta, é integrado o "humano no ciclo" (*human-in-the-loop*) para que usuários experientes sinalizem erros de classificação, o que permite refinar a acurácia do algoritmo em tempo real.

5 DESENVOLVIMENTO DO MÓDULO CLIENTE

A implementação da extensão de navegador possui base nos padrões modernos de segurança e desempenho exigidos pelo ecossistema do *Google Chrome*. Tal abordagem busca evidenciar as escolhas arquiteturais que viabilizam a detecção contínua sem comprometer a experiência do usuário. Referente à fundamentação sobre o *Manifest V3*, a lista de verificação de migração disponibilizada pelo *Chrome for Developers* (Google, 2024) serviu como principal referência técnica.

5.1 Adoção do Padrão Manifest V3 e Separação de Responsabilidades

A escolha pelo *Manifest V3* não é apenas uma conformidade técnica, mas uma decisão tomada para garantir a longevidade e segurança do artefato. A arquitetura do *front-end* foi desenvolvida com base no princípio da estrita separação de responsabilidades, o que se alinha à política de segurança de conteúdo (CSP) restritiva do novo padrão:

- **Estrutura e Estilo (*popup.html* e *popup.css*):** A interface é definida de forma declarativa, sendo abastecida de lógicas de *script inline*. Esta prática atende à exigência do *Manifest V3* que proíbe a execução de códigos não contidos no pacote da extensão, mitigando vulnerabilidades.
- **Lógica de Eventos (*popup.js*):** Toda a interatividade é encapsulada neste módulo. Ele manipula o DOM e estabelece a comunicação futura com o *Service Worker*, garantindo que a execução seja assíncrona e isolada.
- ***Service Workers (Background Scripts)*:** A extensão utiliza *service_workers* em substituição às páginas de fundo persistentes, o que otimiza o uso de recursos do sistema e executa apenas quando eventos específicos de navegação ocorrem.
- **Permissões Declarativas:** O uso de permissões granulares no *manifest.json* (como *tabs* e *storage*) garante que a extensão acesse apenas os dados estritamente necessários para a análise de *phishing*.

5.2 Arquitetura do Front-end e Interface do Usuário

O front-end da extensão foi projetado seguindo heurísticas de usabilidade de Nielsen, com foco em fornecer feedback imediato sem interromper o fluxo de trabalho do usuário:

- **Interceptação e Extração:** O módulo opera de forma assíncrona, extraindo a URL e o conteúdo textual (DOM) da aba ativa assim que o carregamento é concluído.
- **Alertas Visuais:** Caso a API de back-end retorne um veredito positivo para *phishing*, o ícone da extensão exibe alerta visual, informando o estado de risco do site.
- **Módulo de Crowdsourcing (Feedback):** A interface inclui um mecanismo de “um clique” para que o usuário possa reportar erros (falsos positivos ou negativos). Esse feedback é enviado ao *Google Firestore*, atualizando o banco de dados colaborativo e refinando a acurácia do sistema em tempo real.
- **Desempenho (Latência):** A meta de design é que a interação entre o front-end e o back-end ocorra em menos de 500ms, para garantir que a proteção seja imperceptível na experiência de navegação do usuário.

6 DESENVOLVIMENTO BACK-END

6.1 Validação da Infraestrutura de Processamento e Migração de Dados

Conforme descrito na arquitetura do sistema, o núcleo do motor de busca vetorial foi previamente desenvolvido em ambiente local. Nesta etapa atual da pesquisa, o esforço de engenharia foi concentrado estritamente na preparação da infraestrutura em nuvem, especificamente na migração da base de dados local para o *Google Firestore*.

Adicionalmente, foi implementada uma restrição dimensional heurística no algoritmo de ingestão para garantir a viabilidade técnica no ecossistema *NoSQL*: termos cujas listas de adjacência ultrapassavam 15.000 registros de documentos (postings) foram categorizados como excessivamente densos e suprimidos do *upload*. Essa decisão foi baseada no fato de que termos com altíssima frequência em um lote de 100.000 documentos tendem a ser *stop-words* ou vocábulos genéricos, possuindo baixo poder discriminatório para o cálculo da Similaridade do Cosseno. O trecho de código 1 demonstra a implementação desta condição na rotina do consumidor (*worker*).

Para contornar essas limitações e otimizar o tempo de *upload*, foi desenvolvido um *script* de migração dedicado em *Python*, utilizando processamento assíncrono para distribuir a carga de inserção de dados no banco. Adicionalmente, foi implementada uma restrição dimensional

Algorithm 1: Processamento e Ingestão de Termo Individual

Require: *termo, num_postings, data, start_task*

Ensure: *termo* ou *None*

```
1: if num_postings > MAX_POSTINGS_LIMIT then
2:   status  $\leftarrow$  "PULADO_PESADO"
3:   proc_time  $\leftarrow$  (time() – start_task)  $\times$  1000
4:   Escrever linha em METRICS_FILE com status
5:   return None
6: end if
7: doc_ref  $\leftarrow$  db.collection('reputacao_urls_v2').document(termo)
8: await doc_ref.set(data)
9: status  $\leftarrow$  "SUCESSO"
10: proc_time  $\leftarrow$  (time() – start_task)  $\times$  1000
11: Escrever linha em METRICS_FILE com status
12: return termo
```

heurística no algoritmo de ingestão: termos cujas listas de adjacência ultrapassavam 15.000 documentos (*postings*) foram categorizados como excessivamente densos e suprimidos do *upload*. O Trecho de Código 2 demonstra o algoritmo utilizado na rotina do consumidor (*worker*).

Algorithm 2: Upload em Lote do Índice Invertido

Require: *index_data*

Ensure: Execução assíncrona das tarefas de ingestão

```
1: tasks  $\leftarrow$  []
2: for all (i, (termo, postings)) em enumerate(index_data.items()) do
3:   if i  $\geq$  MAX_TERMS_LIMIT then
4:     break
5:   end if
6:   start_task  $\leftarrow$  time()
7:   tasks.append(process_term(termo, len(postings), data, start_task))
8: end for
9: await asyncio.gather(*tasks)
```

A construção do índice invertido para *datasets* massivos foi implementado utilizando a técnica *SPIMI* (*Single-pass in-memory indexing*). O processo fragmenta a indexação em blocos temporários gravados em disco, realizando um *K-way merge* final. Esta abordagem evita o esgotamento de memória RAM (*Out of Memory*) comum em modelos que mantêm o dicionário inteiro em memória. Além disso, o pipeline de ingestão adotou uma política de *Fail-Fast* linguística: caso os recursos do NLTK falhem, o documento é isolado e logado como erro, impedindo a corrupção silenciosa dos cálculos de TF-IDF por falta de *stemming*, porém o processo de ingestão permanece em execução.

Essa estratégia de corte não apenas impediu falhas nas requisições por excesso de carga útil, mas também reduziu drasticamente a volumetria transacionada e o tempo total de processamento, descartando vocábulos genéricos que pouco contribuiriam para a capacidade discriminatória do Modelo Vetorial.

A execução bem-sucedida dessa rotina permitiu que a infraestrutura fosse capaz de suportar e organizar a alta densidade do índice invertido. A consolidação dessa etapa de persistência atua como o pré-requisito fundamental para a arquitetura da solução, assegurando a viabilidade técnica para a futura implementação da API em *FastAPI*, que consumirá esses dados estruturados para a efetiva aplicação do modelo TF-IDF e Cálculo da Similaridade do Cosseno na classificação de ameaças em tempo real.

6.2 Validação da Persistência e Densidade do Índice

A validação da carga de dados no ambiente *cloud* ocorreu por meio da monitorização do processo de ingestão do índice invertido. Esta etapa consiste em um processo *offline* de escrita em lote (*batch upload*) para popular o banco de dados. Dessa forma, os tempos de resposta registrados se referem exclusivamente às transações de gravação no *Google Firestore*, não possuindo relação direta com a latência de leitura da API, cuja meta de desempenho permanece inferior a 500 milissegundos.

Para garantir a estabilidade operacional e respeitar o limite de 20.000 escritas diárias do plano gratuito do Google Firebase, a migração inicial foi limitada a um vocabulário de 19.500 termos únicos. É importante distinguir que, enquanto o filtro de 15.000 *postings* suprime termos excessivamente densos (prováveis *stop-words*) para evitar que documentos excedam 1 MB no *Firestore*⁵. Já o limite de 19.500 termos define a amplitude total do dicionário nesta etapa experimental. Esta estratégia assegura a viabilidade do sistema sem comprometer a eficácia do Modelo Vetorial.

7 RESULTADOS E DISCUSSÃO

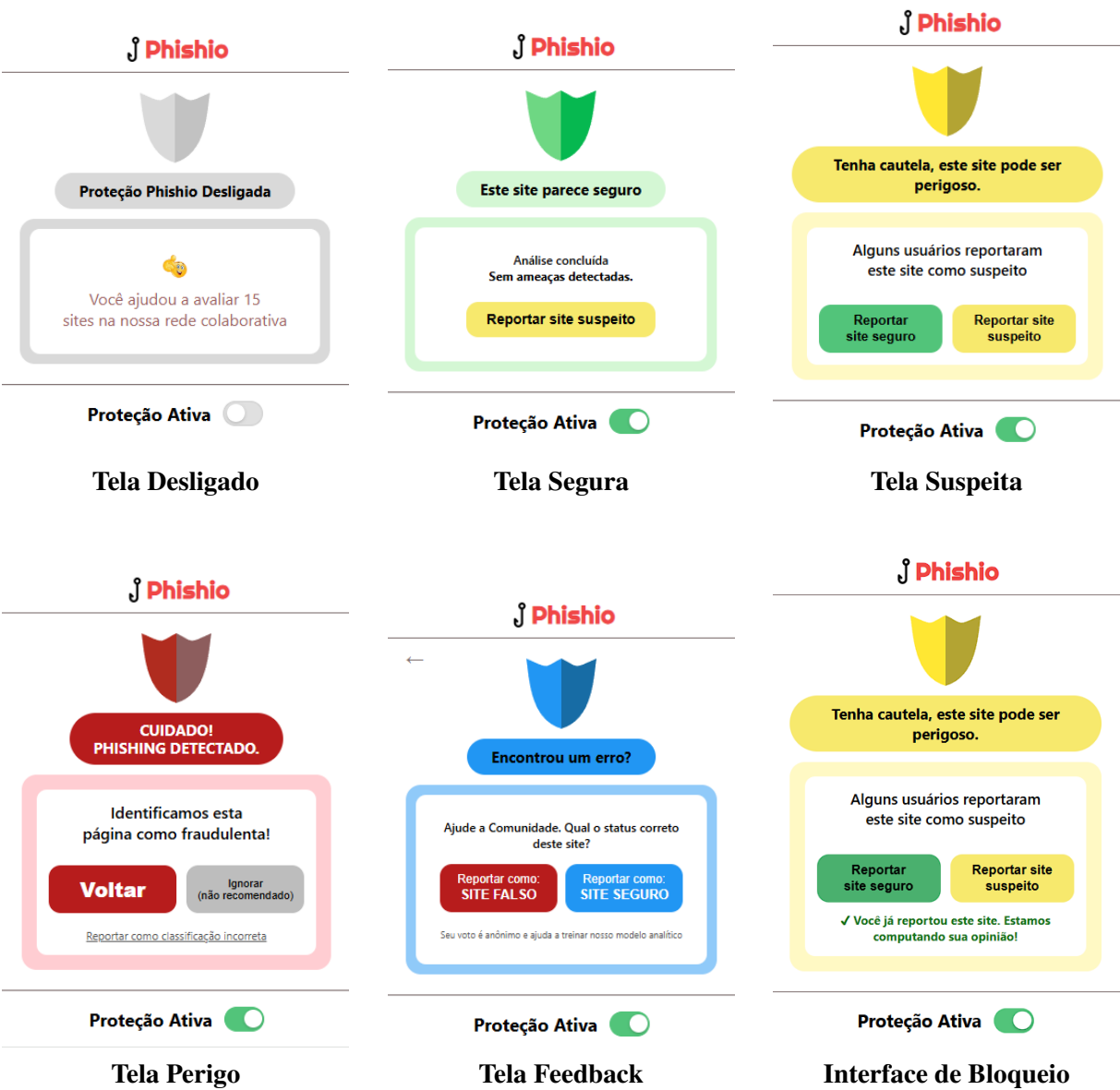
O projeto avançou significativamente na construção de sua arquitetura, foi possível realizar a entrega protótipo funcional da extensão para o navegador Google Chrome e de sua respectiva infraestrutura em nuvem. Como o sistema se encontra em fase de testes de integração e refinamento de *software*, os indicadores a seguir apresentam o panorama atual e os resultados preliminares da solução.

⁵O *Google Cloud Firestore* impõe uma restrição arquitetural rigorosa (*hard limit*) de 1MB para o tamanho máximo de um único documento (Google Firebase, 2026). No modelo de modelagem *NoSQL* adotado neste projeto, as listas de adjacência (*postings*) são embutidas diretamente como atributos do documento de cada termo. Consequentemente, indexar vocábulos genéricos de altíssima frequência geraria matrizes extensas, estourando esse limite estrutural e provocando a falha (rejeição) da transação de escrita por parte da API do banco de dados.

7.1 Desenvolvimento do front-end e back-end

Referente ao desenvolvimento do *front-end*, houve a entrega do protótipo visual dinâmico exibido no conjunto de figuras 4, estruturado em cinco painéis de estado orientados por cores semânticas. O sistema contempla as telas de inatividade (cinza), navegação segura (verde), alerta de suspeita com base em reportes da comunidade (amarelo), bloqueio de ameaça de *phishing* detectada (vermelho) e o módulo interativo de *feedback* (azul). A materialização dessas interfaces valida a aplicação prática das heurísticas de usabilidade, garantindo clareza e interatividade para o engajamento do usuário.

Figura 4 – Interface de usuário da extensão Phishio: estados de proteção, alertas de segurança e mecanismos de feedback colaborativo.



Fonte: Dados da Pesquisa (2026).

A validação experimental demonstrou que a API manteve a integridade dos dados mesmo

sob múltiplas requisições simultâneas. O acesso ao arquivo binário de 4,5 GB foi protegido por um mecanismo de *Lock* de concorrência, impedindo a sobreposição de ponteiros de leitura (*seek/read*) que causava falhas no motor vetorial em versões anteriores.

Além disso, o sistema passou a fornecer *feedback* de estado em tempo real. Durante o processamento da Similaridade do Cosseno, a extensão exibe um *badge* informativo ("...") no ícone do navegador, atendendo às heurísticas de visibilidade do estado do sistema de Nielsen.

Figura 5 – Popup exibido na barra de extensões enquanto o sistema checa a URL e o DOM



Fonte: Dados da Pesquisa (2026).

Do lado do back-end, foi construída uma lógica de detecção colaborativa, o qual colaborou com a implementação da base arquitetural para o objetivo de uso social do sistema, incluindo a criação do *endpoint* de recepção de requisições (*/reportar_url*). Atualmente, o módulo está em fase de depuração (*debugging*) para garantir a sincronização exata, de ponta a ponta, entre o voto emitido pelo usuário na extensão (através do painel de *feedback* azul) e o registro transacional no banco de dados colaborativo (*Google Firestore*), buscando assegurar a precisão no cálculo final do consenso.

A integração do motor de busca (*SearchEngine*) demonstrou a viabilidade prática do uso de Recuperação de Informação na segurança *web*. Testes iniciais comprovam a capacidade estrutural da API em processar o conteúdo da página (DOM), calcular a ponderação TF-IDF e aplicar a Similaridade de Cosseno, estabelecendo a base para mitigar ameaças *zero-hour*. Entretanto, nesta etapa, ficou evidenciada uma dificuldade conhecida: os falsos positivos. Essa barreira pode ser superada por meio da ação colaborativa dos usuários, visto que o sistema também considera os votos da comunidade para classificar as páginas *html*.

Quanto ao desempenho da interface e latência, a estratégia de arquitetura híbrida se mostrou tecnicamente promissora. O roteamento que prioriza a consulta ao *cache* de reputação orienta as atuais otimizações de código, mantendo como meta primária um tempo de resposta imperceptível ao usuário, estritamente abaixo de 500 milissegundos ($< 500ms$).

Sobre distribuição geográfica e observabilidade, a infraestrutura configurada no *back-end* já estabelece a capacidade técnica para a interceptação segura e captura anonimizada de endereços IP. Este recurso prepara o terreno para futuras análises retrospectivas e o mapeamento da concentração regional de ataques.

A seguir, são apresentados os resultados obtidos a partir da execução do sistema em ambiente de produção (*Cloud*), com a avaliação do desempenho do motor vetorial e da eficácia das camadas de detecção frente à diferentes cenários de navegação.

7.2 Ambiente de Testes e Metodologia

A validação experimental foi realizada em uma instância virtual (*Droplet*) na plataforma *DigitalOcean*⁶, operando com sistema *Ubuntu 24.04*, 4 GB de memória RAM e 80 GB de armazenamento em SSD. A comunicação entre a extensão de navegador e a API utilizou o protocolo HTTPS (TLS 1.3).

A avaliação foi conduzida por meio do *framework Pytest*⁷, totalizando 27 casos de teste automatizados divididos em categorias unitárias, de caixa-preta, de segurança e de casos de borda (*edge cases*). A suíte de testes apresentou um tempo de execução de 9,04 segundos, atingindo uma cobertura de código de 73% (123 de 160 linhas executadas no módulo principal) utilizando a ferramenta *Coverage.py*⁸. As linhas não cobertas concentram-se em blocos de tratamento de exceções para falhas críticas de infraestrutura e no *script* de ingestão de dados *offline* (*ingest_firestore.py*), que opera independentemente do fluxo de tempo real da API. Os resultados são apresentados em formato de tabela no terminal após a execução, como mostrado na figura 6

Figura 6 – Resultados do Teste de Cobertura

```
===== tests coverage =====
----- coverage: platform win32, python 3.12.10-final-0 -----
```

Name	Stmts	Miss	Cover	Missing
runtime\core\Config.py	12	0	100%	
runtime\core\Geolocator.py	28	19	32%	14-18, 26-49
runtime\core\Linguistic.py	28	8	71%	12-13, 17-18, 29, 31-33
runtime\core\SearchEngine.py	142	26	82%	32-34, 46-48, 56, 69-72, 83-85, 91, 112, 119-121, 135, 167-173, 177, 187
runtime\ingest_firestore.py	30	30	0%	2-45
runtime\main.py	160	106	34%	50, 60-62, 115-123, 129-224, 236-251, 260-307, 325-352
TOTAL	400	189	53%	

```
===== 3 passed in 4.84s =====
```

Fonte: Dados da Pesquisa (2026).

7.2.1 Limiar de Alerta e Critérios de Decisão

O sistema utiliza o Modelo Vetorial para o cálculo da Similaridade do Cosseno entre o conteúdo capturado e o índice de assinaturas. A lógica de decisão foi configurada com três níveis de classificação baseados no *score* resultante (*sim*). O posicionamento dos cenários de teste frente a esses limiares é apresentado na Figura 7.

- **Seguro (Safe):** $sim < 0,40$. O conteúdo não apresenta similaridade estatística com padrões maliciosos conhecidos.
- **Suspeito (Suspicious):** $0,40 \leq sim < 0,75$. Indica sobreposição de termos de isca, disparando o alerta amarelo.

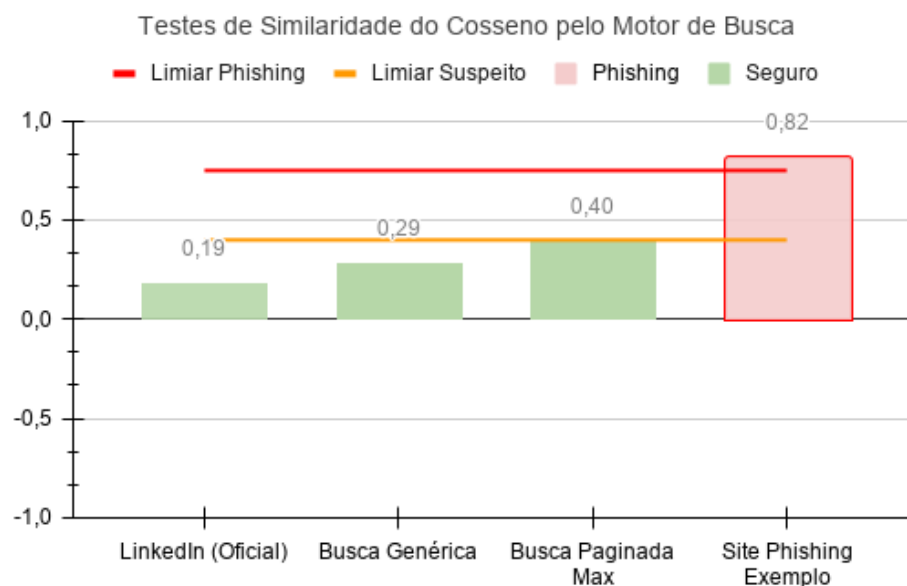
⁶Disponível em: <https://www.digitalocean.com/>. Acesso em: 02 de Mai. de 2026.

⁷Disponível em: <https://docs.pytest.org/en/stable/getting-started.html>. Acesso em: 20 de Mai. de 2026

⁸Disponível em: <https://coverage.readthedocs.io/en/7.13.5/>. Acesso em: 02 de Mai. de 2026.

- **Phishing:** $sim \geq 0,75$. A similaridade técnica justifica o bloqueio imediato da URL.

Figura 7 – Distribuição de scores e classificação pelo motor de busca



Fonte: Dados da Pesquisa (2026).

Nos testes de integração, o motor processou o site *LinkedIn* identificando 76.091 documentos candidatos em 0,3669 segundos, com score de 0,1855. Em buscas paginadas, o sistema retornou resultados em 0,0853 segundos com score máximo de 0,3982, ficando no limite superior da zona de segurança. Em ambos os cenários, os tempos de resposta mantiveram-se abaixo do requisito de latência de 500ms definido na arquitetura.

8 CRONOGRAMA

Para viabilizar o desenvolvimento da arquitetura proposta e a condução da pesquisa experimental, as atividades foram organizadas em etapas sequenciais, alinhadas aos marcos avaliativos do semestre letivo. A Tabela 1 detalha a distribuição das tarefas ao longo dos meses de execução do projeto.

Tabela 1 – Cronograma de atividades do Trabalho de Conclusão de Curso

Atividades	Fev	Mar	Abr	Mai	Jun	Jul
Definição da arquitetura e elaboração da Proposta (Entrega 1)	X	X				
Desenvolvimento da API (<i>Back-end</i>) - MVP para 13/03	X	X				
Integração em nuvem e Extensão Manifest V3 (<i>Front-end</i>)		X	X			
Testes experimentais e Entrega Intermediária (Entrega 2)			X	X		
Instrumentação, coleta de dados e refinamento do algoritmo				X	X	
Elaboração da apresentação (Slides)				X	X	
Redação final do artigo (Entrega 3)				X	X	
Defesa perante a banca examinadora					X	X

Fonte: Dados da Pesquisa (2026).

REFERÊNCIAS

- APWG. **Phishing Activity Trends Report: 4th Quarter 2024**. [S.l.], 2025. Acesso em: 19 fev. 2026.
- BAEZA-YATES, R. *et al.* Modern information retrieval. 07 1999.
- Federal Bureau of Investigation. **Internet Crime Report 2024**. Washington, D.C., 2024. Acesso em: 19 fev. 2026.
- FETTE, I.; SADEH, N.; TOMASIC, A. Learning to detect phishing emails. In: **Proceedings of the 16th international conference on World Wide Web**. [S.l.: s.n.], 2007. p. 649–656.
- Google. **Lista de verificação de migração do Manifesto V3**. 2024. <<https://developer.chrome.com/docs/extensions/develop/migrate/checklist?hl=pt-br>>. Acesso em: 25 de fevereiro de 2026.
- Google Firebase. **Cloud Firestore pricing**. [S.l.], 2026. Disponível em: <<https://cloud.google.com/firestore/pricing>>. Acesso em: 23 mar. 2026.
- IBM. **Cost of a Data Breach Report 2025**. Nova York, 2025. Acesso em: 19 fev. 2026. Disponível em: <<https://www.ibm.com/reports/data-breach>>.
- KOLESNIKOV, A.; CUOZZO, L. Análisis del ciclo de vida de las páginas de phishing y de estafas. **Securelist by Kaspersky**, 2021. Acesso em: 19 fev. 2026. Disponível em: <<https://securelist.lat/phishing-page-life-cycle/95951/>>.
- MARTÍNEZ, G. Phishing: Three key trends and how to protect yourself. **LACNIC Blog**, mar. 2025. Acesso em: 19 fev. 2026. Disponível em: <<https://blog.lacnic.net/en/cybersecurity-smishing-attacks/>>.
- NIELSEN, J. **10 Usability Heuristics for User Interface Design**. Nielsen Norman Group, 1994. Disponível em: <<https://www.nngroup.com/articles/ten-usability-heuristics/>>. Acesso em: 20 fev. 2026.
- SQLite Development Team. **SQLite Documentation**. [S.l.], 2026. Disponível em: <<https://www.sqlite.org/docs.html>>. Acesso em: 20 abr. 2026.
- SUROWIECKI, J. **A sabedoria das multidões**. [S.l.]: Lua de Papel, 2023.
- TIANGOLO. **FastAPI framework, high performance, easy to learn, fast to code, ready for production**. 2026. Disponível em: <<https://fastapi.tiangolo.com/>>. Acesso em: 20 abr. 2026.
- Verizon. **2024 Data Breach Investigations Report (DBIR)**. Nova York, 2024. Acesso em: 19 fev. 2026. Disponível em: <<https://www.verizon.com/business/resources/reports/dbir/>>.
- ZHANG, Y.; HONG, J. I.; CRANOR, L. F. Cantina: a content-based approach to detecting phishing web sites. In: **Proceedings of the 16th International Conference on World Wide Web**. New York, NY, USA: Association for Computing Machinery, 2007. (WWW '07), p. 639–648. ISBN 9781595936547. Disponível em: <<https://doi.org/10.1145/1242572.1242659>>.